SITAS

SYSTEM AND RUNTIME DOCUMENTATION

Frode Randers

2026-05-19

Contents

1 Introduction

1.1 Purpose

`sitas` is an experimental Rust runtime and service model inspired by Seastar's shard-per-core architecture. It is not a direct clone. The project is about understanding the runtime internals: how work is owned by shards, how cross-shard messages move, how a small executor drives futures, and how OS completion and readiness mechanisms can be exposed without depending on a third-party async runtime.

The code base started with a standard-library-only sharded key-value store. The current branch extends that baseline with a custom executor, Unix readiness primitives, TCP helpers, sharded async execution, shard-local values, observability snapshots, CPU placement, and experimental Linux `io_uring` support.

1.2 How to read this project

The easiest way to understand `sitas` is to read it from the outside in, then return from the inside out.

From the outside, the important idea is that application code should see typed, owned boundaries. A caller asks a service or a shard to do work. The runtime decides which shard owns that work. The owning shard mutates its local state and returns an owned reply. This is the same mental shape as a service call, but the implementation deliberately avoids hiding service state behind a global mutex.

From the inside, the interesting question is how the runtime keeps that model honest:

1. futures are owned by executor tasks;
2. tasks are assigned to one executor shard;
3. wakers only requeue tasks on that executor;
4. cross-shard work must be submitted explicitly;
5. shard-local values can be borrowed only inside synchronous closures;
6. observability returns copies, not borrowed internals.

This manual follows that path. The architecture chapter explains the ownership rules. The executor chapter explains how futures become runnable work. The sharding chapter explains how one executor becomes many independent shards. The I/O chapter explains why readiness and completion are different mechanisms and how both appear in the runtime.

> **Learning goal.** If you can explain where a future is polled, who owns the state it mutates, what wakes it, and what happens when it is dropped, you understand the most important parts

of this code base.

1.3 Why the project has this shape

The runtime is intentionally built from small visible pieces because the project is as much about explanation as about capability. A mature runtime can hide task allocation, reactor registration, cross-thread wakeups, cancellation, and kernel ownership behind polished APIs. That is useful in production, but it is not ideal when the goal is to understand the machinery.

`sitas` therefore accepts some explicitness that a general-purpose runtime might smooth over:

Explicit shards make ownership visible. If work runs on shard 2, shard 2 owns the mutation.
Explicit submitters make cross-shard movement visible. A future does not silently migrate just because another shard is less busy.
Explicit snapshots make observation visible. Monitoring is a read of owned runtime facts, not a borrowed view into live internals.
Explicit OS boundaries make platform behavior visible. A readiness wait, an `io_uring` completion, and a CPU affinity request are different contracts with the operating system.

This is the main design tradeoff in the code base. The project prefers APIs that teach the boundary they cross over APIs that hide the boundary too early.

1.4 Scope of this manual

This manual is a LaTeX companion to the Markdown architecture notes. It is intended for longer-form system documentation and for keeping a printable view of the evolving runtime design.

It focuses on:

- architectural invariants;
- executor and scheduling behavior;
- shard ownership and cross-shard submission;
- readiness, `epoll`, and `io_uring` integration;
- example programs that exercise the runtime;
- build and validation operations.

> **Note.** The project is still deliberately experimental. APIs are allowed to move when that exposes cleaner runtime internals or a better teaching surface.

1.5 What this project is not

`sitas` is not trying to become a production replacement for mature Rust runtimes. It is also not trying to copy Seastar line by line. The point is to keep the mechanisms small enough that they can be inspected:

- no Tokio, async-std, Glommio, Monoio, or actor framework is used as the core runtime;
- blocking standard-library service APIs remain distinct from awaitable executor APIs;
- unsafe and FFI code is kept near the operating-system boundary;
- mechanisms are introduced only after their semantics can be tested.

This makes the project useful as a learning resource. It has enough machinery to demonstrate timers, wakers, readiness, `io_uring`, shard-local values, scheduling groups, and CPU placement, while still leaving those mechanisms visible in ordinary Rust modules.

2 Architecture

2.1 Invariants

The project is guided by a small set of invariants:

1. Each piece of application service state is owned by one shard.
2. Only the owning shard may mutate its service state.
3. Application services should not be modeled as shared `Arc<Mutex<Service>>` state.
4. Cross-shard interaction happens through typed messages, typed reply handles, or typed submitter calls.
5. Values crossing shard boundaries are owned.
6. References to shard-local state must not escape synchronous access closures and must not cross `.await`.
7. Async work remains on its assigned shard unless explicitly submitted elsewhere.
8. Observability snapshots are owned values, not borrowed runtime internals.
9. Unsafe code is isolated behind small safe APIs.

2.2 The shared-nothing rule

The central design rule is simple: application state has one owner. In the standard-library services that owner is a shard thread receiving typed commands. In the async runtime exploration that owner is an executor shard polling futures assigned to it. The form changes, but the rule is the same.

This is why the project avoids turning service state into `Arc<Mutex<Service>>`. A mutex would make mutation mechanically safe, but it would hide the runtime question that the project is trying to study: which shard owns this state, where does work run, and when does the ownership boundary get crossed?

The shared-nothing model does not mean the runtime never uses synchronization. It does. Wakers, reply state, lifecycle flags, counters, and submission queues are runtime machinery. The important distinction is that application service state is not normally protected by those locks. It stays local to its shard.

> **Why avoid a service mutex?.** A service mutex answers only one question: can two threads mutate this value at the same time? The runtime is asking a stronger question: which shard is responsible for this value, which work is allowed to touch it, and how does another shard request a change? Keeping service state shard-owned preserves that answer in the program structure.

```
Caller thread or task
    |
    | owned command / owned future / typed submitter call
    v
Owning shard
    |
    | synchronous local mutation
    v
Owned reply or owned snapshot
```

Listing 2.1: State ownership model

2.3 Boundaries and lifetimes

Most runtime bugs in systems like this are boundary bugs. A value crosses to another thread without being owned. A reference to local state escapes a closure. A future keeps a borrowed value across `.await`. A completion operation drops a buffer before the kernel has finished with it.

`sitas` tries to make those boundaries visible:

Cross-shard transfer uses owned values and explicit `Send + 'static` requirements where a future can move to another thread.
Shard-local access gives a closure synchronous access to `&mut T`; the closure returns an owned result and cannot suspend.
Observability returns owned snapshots. A monitoring thread can keep a snapshot after the runtime has moved on.
Kernel I/O stores owned buffers until completions are observed or until abandoned operation cleanup proves they can be discarded.

2.4 Why owned snapshots

Observability could expose references into executor internals, but that would make a diagnostic API participate in the same lifetime and locking problems as the runtime itself. The project instead treats observation as a copy boundary. A snapshot is a point-in-time fact value that can be sorted, printed, diffed, or sent to another thread without keeping the executor borrowed.

This matters because long-running runtime work needs inspection while it is still changing. Owned snapshots let monitoring code be ordinary application code. They also prevent a debugging tool from accidentally extending the lifetime of task, timer, or shard-local internals.

2.5 Layered model

The current runtime can be read as a layered system:

```
Application service APIs
    |
Typed command/reply and submitter APIs
    |
Placement and shard routing
    |
Shard-local service state / ShardLocal<T>
    |
Shard executor or std shard thread
    |
Mailbox, task queue, timers, readiness, wakeups
    |
OS primitives: pipe/poll/epoll/io_uring/affinity
```

Listing 2.2: Conceptual layers

The standard-library baseline exercises the upper part of the model. The non-std runtime branch fills in the executor and OS layers while preserving the same shared-nothing service direction.

2.6 Primary modules

`runtime` The reusable standard-library kernel for bounded shard mailboxes, reply handles, shard startup, and shutdown.
`kv` The reference sharded key-value service.
`counter` A smaller second service proving that the runtime primitives are not key-value-specific.
`placement` Key-to-shard routing.
`os` Unix FFI, wake pipes, readiness backends, affinity, and Linux `io_uring` experiments.
`executor` The single-threaded async kernel.
`sharded_executor` One executor per shard thread, plus submission and observation APIs.
`shard_local` One owned value per executor shard with checked local access.

2.7 How to trace a feature through the layers

When learning or extending the runtime, trace one feature down the stack. For example, a named task in a scheduling group passes through several layers:

1. user code calls a typed spawn API;
2. the spawner validates that the scheduling group belongs to this executor;
3. a task is allocated with a name and group id;
4. the scheduler places the task into the group's ready queue;
5. the executor polls the selected task;
6. the task records status, wait interest, poll count, and timestamps;
7. a snapshot later resolves the group id back to an owned group name.

This kind of trace is more useful than reading modules in isolation. It shows which layer owns validation, which layer owns scheduling, which layer owns observability, and where the public API remains safe.

3 Executor

3.1 Role

The executor is a minimal single-threaded async kernel. It owns pinned futures inside tasks, maintains a ready queue, coalesces repeated wakes, and drives a root future through `run_until` while also polling spawned tasks.

The executor deliberately avoids Tokio or another third-party runtime. That keeps the interesting internals visible: wakers, task queues, timers, readiness registration, cancellation, and completion dispatch all live in this code base.

3.2 Why a custom executor

The custom executor exists because the project needs to control the meaning of "where work runs." In a shard-per-core runtime, a future is not just an async computation; it is also part of a placement decision. The executor must be able to say that this task belongs to this shard, this scheduling group, this ready queue, and this local reactor state.

A third-party runtime would solve many engineering problems, but it would also move the most interesting questions behind someone else's scheduler. Here the small executor is the laboratory. It makes it possible to inspect how a waker requeues one task, how a dropped timeout removes timer state, how a TCP future records fd interest, and how a completion future keeps buffers alive.

3.3 Responsibilities

- Poll ready tasks with a budget so self-waking tasks cannot starve timers and I/O forever.
- Implement custom task wakers that enqueue tasks onto the local ready queue.
- Catch task panics at executor boundaries.
- Preserve panic payloads where join handles and `block_on` make them observable.
- Provide `sleep`, `timeout`, `race`, and `yield_now`.
- Provide typed join handles and abort support.
- Provide cooperative stop tokens, `Notify`, and `TaskScope`.
- Drive readiness futures for read and write interests.
- On Linux, drive executor-owned `io_uring` read/write-at futures.

3.4 Task lifecycle

A spawned future is stored inside a task. The task has a stable executor-local `TaskId`, an optional name, a scheduling group id, and internal state describing whether it is queued, polling, waiting, completed, or cancelled.

The lifecycle is intentionally explicit:

```
spawn
  -> create Task with pinned Future
  -> queue Task in a scheduling group
  -> poll Future
       Ready(())  -> completed
       Pending    -> waiting until waker fires
  -> wake
  -> queue Task again
```

Listing 3.1: Simplified task lifecycle

A repeated wake does not create repeated queue entries for the same task. The task records that it is already queued, so repeated wakeups coalesce. This is a small but important executor property: a chatty future should not multiply itself in the ready queue.

Cancellation is normal future dropping. Aborting a join handle or dropping an executor eventually drops the owned future. Timer and readiness futures must therefore clean up their registrations in `Drop`; otherwise the reactor would keep stale interests for tasks that no longer exist.

> **Why cancellation is treated as ordinary.** Rust futures are cancelled by being dropped. The runtime cannot rely on a separate cancellation callback running later. Cleanup must therefore be owned by the future, task, join handle, or dispatcher state that is being dropped. This is why many tests exercise timeout, abort, and dropped-future paths rather than only successful completion.

3.5 Wakers and the ready queue

The custom waker does one job: make the task runnable again on the same executor. It does not move the task to another shard, does not poll the task directly, and does not borrow application state. It marks the task queued and hands it back to the scheduler.

The executor loop then decides when to poll ready work. It does so in batches bounded by `READY_POLL_BUDGET`. That budget is a fairness valve. A self-waking task can keep itself ready, but it cannot prevent the executor from checking timers, readiness, and completions forever.

The ready queue is intentionally executor-local. Cross-thread wakeups may need to notify an executor that work became ready, but the task itself is still polled by the executor that owns it. This keeps local task state, scheduling group accounting, and reactor registrations aligned.

3.6 Scheduling groups

Scheduling groups are the first resource-class mechanism in the executor. Each group owns a ready queue and has a relative share count. Ordinary spawn calls use the default group. Explicit group APIs let code classify work such as foreground request handling and background maintenance.

The reason to add groups before more advanced load balancing is that resource classification is a local scheduling problem. A shard can decide how to divide its own polling time among foreground and background tasks without moving application state to another shard. That makes scheduling groups a natural fit for the shared-nothing model.

The selection rule is deliberately simple. The scheduler chooses a non-empty group with the lowest weighted virtual runtime, pops one task from that group's FIFO queue, polls it, and charges the group for the observed wall-clock poll duration. The charge is scaled by the group's shares.

```
for each poll:
    group = non_empty_group_with_lowest_virtual_runtime()
    task = group.pop_front()
    duration = poll(task)
    group.virtual_runtime += duration * default_shares / group.shares
```

Listing 3.2: Weighted group selection

This is not preemption. A long poll still blocks the executor thread until the future returns `Poll::Pending` or `Poll::Ready`. The mechanism is cooperative: futures must yield, await I/O, await timers, or otherwise return control to the executor.

Scheduling group handles are executor-local. A group created by one executor is rejected by another. The default group handle is special because it names the built-in default queue in any executor. The same idea appears in the sharded runtime as a `ShardedSchedulingGroup`, which is one executor-local group per shard plus a runtime identity check.

3.7 Structured child tasks

`TaskScope` groups child tasks under one owner. Dropping the scope requests cooperative stop and aborts still-owned children. Calling `shutdown` asks children to stop and waits for them. Calling `shutdown_timeout` bounds that wait and aborts uncooperative children.

Scopes can also spawn children into scheduling groups. This matters when a server accepts work that should be both lifetime-scoped and resource-classified. The scope does not implement scheduling itself; it delegates group validation and task creation to the underlying `Spawner`.

3.8 Wait ordering

Each loop iteration polls ready work, wakes expired timers, dispatches locally available `io_uring` completions, and then decides how to wait.

On Linux, pending `io_uring` submissions are first submitted to the kernel without blocking. The executor then includes the ring descriptor in the same reactor wait as read interests, write interests, reactor wakes, and the next timer deadline. When the ring descriptor becomes readable, the dispatcher harvests completions and wakes the registered task wakers.

This gives the executor one idle wait shape instead of a separate `io_uring`-first wait path. The important semantic point is not that `io_uring` has become readiness I/O; it has not. Completion ownership and buffer lifetime are still handled by the dispatcher. The reactor only tells the executor that completion queue progress is available.

3.9 Observability

Executor snapshots are owned values. They include task counts, named task state, polling counters, timer counts, readiness counts, and Linux `io_uring` dispatcher counters when the dispatcher is installed.

Task snapshots include the task name, scheduling group id and name, lifecycle state, wait interest, poll count, accumulated poll time, and key timestamps. Helper methods derive task age and current state duration from a caller-supplied `Instant`. This keeps the snapshot itself factual while making progress views easy to write.

```
task name:               useful for humans
status:                  queued, polling, waiting, completed, cancelled
waiting_for:             timer, readable fd, writable fd, unknown, none
poll_count:              how many times the future was polled
total_poll_time:         how much executor time the task consumed
state_duration_at(now): how long it has been in the current state
```

Listing 3.3: Reading a task snapshot

4 Sharding

4.1 Shard-per-core direction

`ShardedExecutor` connects the single-threaded executor to the shard-per-core model. It starts one executor on each shard thread and provides APIs for submitting work to one shard, all shards, or a map/reduce over shards.

The model is explicit: work does not migrate implicitly. A future runs on its assigned shard until code deliberately submits work elsewhere.

This mirrors the most important Seastar idea without copying its C++ API: parallelism comes from many mostly-independent shards, not from many threads mutating the same service object. Each shard has its own executor, task queues, timers, readiness registrations, and shard-local values.

The reason for this shape is cache locality and ownership clarity. If each shard owns its state and mostly works on its own queue, the runtime does not need a global lock around application data. Cross-shard communication still exists, but it becomes an explicit cost in the design instead of an accidental side effect of shared references.

```
Shard 0 thread        Shard 1 thread        Shard 2 thread
  Executor              Executor              Executor
  local tasks           local tasks           local tasks
  local timers          local timers          local timers
  local state           local state           local state
```

Listing 4.1: Shard-per-core runtime shape

4.2 Cross-shard submission

`ShardedSubmitter` is the explicit cross-shard async mechanism. A task on one shard can submit a future to another shard and await the returned join handle. The remote future is polled on the target shard. The awaiting task resumes on its original shard after the remote work completes.

Submitters own spawner clones. They are lifetime capabilities and must be dropped before a runtime can fully drain.

The important detail is the return path. A task on shard 0 can submit work to shard 1 and await the join handle. The submitted future is polled on shard 1. When it completes, the awaiting future on shard 0 is woken and continues on shard 0. The result crosses the boundary as an owned value.

```
Shard 0 task
    submit future to Shard 1
    await JoinHandle

Shard 1 executor
```

```
    poll submitted future
    complete owned result

Shard 0 task
    wakes and receives result
```

Listing 4.2: Cross-shard async flow

This is why cross-shard APIs require `Send + 'static` where a future or value can move to another thread. It is also why references into shard-local state cannot be submitted.

The submitter API is deliberately more explicit than a general work-stealing pool. Work stealing would be interesting later, but it would blur the lesson this runtime is trying to preserve: placement is part of the program's semantics. When code submits to shard 1, it is saying that shard 1 is the correct owner for that unit of work.

4.3 Shard-local values

`ShardLocal<T>` owns one value per executor shard. Access is routed to the owning shard and the user closure receives `&mut T` synchronously. The implementation uses internal unchecked storage plus runtime owner checks, not a mutex around application state.

The critical rule is that references to local state cannot escape the closure and cannot cross `.await`. This keeps ownership local even when the runtime itself uses synchronization for task bookkeeping.

There are two useful ways to read `ShardLocal<T>`:

- as a safe teaching model for shared-nothing state, because each shard owns exactly one `T`;
- as a runtime boundary test, because the code must prevent references to `T` from becoming async state.

The API therefore prefers closures returning owned results. If a computation needs to wait, split it into two phases: first copy or compute an owned value from the shard-local state, then `await` using that owned value outside the access closure.

> **Why not async closures for local access?.** An async closure can suspend while holding references captured from its environment. That is exactly what shard-local state must avoid. The synchronous closure form is less flexible, but it makes the safe pattern obvious: inspect or mutate local state now, return an owned value, and only then await.

4.4 Sharded scheduling groups

A sharded scheduling group is not a global queue. It is a convenience handle containing one executor-local scheduling group per shard. Creating a group called `foreground` on a four-shard runtime creates four independent executor-local `foreground` groups, all with the same shares.

This preserves the shared-nothing model. Each shard still schedules only its own tasks. The sharded group handle simply lets callers use one typed API to spawn or submit matching classes of work across shards.

Runtime identity matters. A sharded group created by one `ShardedExecutor` is rejected by another runtime, even if both runtimes have a group with the same name. This prevents accidental targeting of same-numbered executor-local group ids in the wrong runtime.

4.5 CPU placement

CPU placement is an experimental Linux-supported runtime request. Linux applies hard affinity with `sched_setaffinity` where supported and observes container cpuset restrictions through `sched_getaffinity`. Non-Linux platforms report unsupported placement honestly.

By default, placement failures are recorded in shard snapshots and startup can still succeed. Required placement turns that into fail-fast startup behavior.

CPU placement is best understood as an explicit request, not as a finished portable scheduler. On Linux, the runtime asks the kernel to pin shard threads to particular CPUs. In containers, the available CPU set may already be restricted, so the runtime first observes what the kernel says is available. On macOS and other unsupported platforms, placement reports unsupported rather than pretending that pinning happened.

The feature is useful even in this experimental form because it forces the runtime to separate desired placement from observed placement. A shard can say which CPU it asked for, which CPU set the host allowed, and whether startup treated failure as advisory or fatal. Those facts are more valuable than a portable abstraction that hides when pinning did not actually occur.

5 I/O Runtime

5.1 Readiness

The readiness path is intentionally small. A future registers interest in a file descriptor, the scheduler records the waker, the Unix reactor sleeps using Linux `epoll`, macOS/iOS `kqueue`, or fallback `poll` plus a wake pipe, and readiness wakes the interested task.

This path is readiness-based, not completion-based. It is used by the current TCP helpers and remains separate from the experimental `io_uring` completion path.

Readiness is the first I/O layer because it matches Unix non-blocking sockets well and it is portable across the platforms used for development. It also keeps ownership simple: the stream or listener remains owned by user code, and the runtime only remembers which task should be woken when the descriptor may make progress.

Readiness means that the operating system says an operation is likely to make progress. It does not mean the runtime has the bytes. A readable TCP stream may still return fewer bytes than requested. A writable stream may accept only part of a buffer. The future must attempt the operation, handle `WouldBlock`, and register interest again when necessary.

```
poll future:
    try non-blocking operation
    if operation succeeds:
        return Ready(result)
    if operation would block:
        register fd interest and waker
        return Pending

reactor wait:
    OS reports fd readiness
    scheduler wakes interested task
```

Listing 5.1: Readiness future shape

This is why readiness futures do not own file descriptors. The caller owns the stream or listener. The future only borrows it for the duration of the async operation and registers interest in the raw descriptor while pending.

That borrowing model is useful for TCP. A connection handler can own a stream and call several helper futures against it in sequence. The reactor does not become the owner of the connection; it is only the waiting mechanism.

5.2 TCP helpers

The TCP helpers are intentionally written on top of readiness futures rather than hiding a separate runtime. They demonstrate the normal async server shape:

1. put the listener or stream in non-blocking mode;
2. accept or connect using readiness futures;
3. spawn one handler task per accepted connection;
4. use `TaskScope` or join handles to wait for handlers;
5. propagate handler errors as ordinary `io::Result` values.

Grouped TCP helper variants classify accepted handler tasks into scheduling groups. The accept loop remains in the caller's task; only the per-connection handler work is moved into the requested group. This is useful for separating foreground request handling from background maintenance or low-priority connections.

This split is important because accepting connections and serving connections have different lifecycle and scheduling needs. The accept loop is usually one long-lived task. Handlers are many shorter-lived tasks whose resource class may depend on the service being provided. Keeping those roles separate makes shutdown, observability, and scheduling group assignment easier to reason about.

> **Readiness lesson.** Readiness is a notification that retrying may be useful. Completion is a notification that the operation has already finished. The code keeps those concepts separate because the ownership and cancellation rules are different.

5.3 `io_uring`

The Linux `io_uring` backend is experimental. It currently supports a narrow executor integration for owned-buffer `read_at`, `read_exact_at`, and `write_at` style file I/O.

`io_uring` is included because it exercises a harder runtime problem than readiness. The kernel can own an operation after the Rust future that submitted it has been dropped. That forces the runtime to model operation ids, owned buffers, late completions, and abandonment. Those are exactly the kinds of internals this project wants to make visible.

Each Linux executor run loop installs a thread-local dispatcher when the host allows ring setup. Executor-backed `io_uring` futures store operation ids and owned buffers. When polled on a shard, they queue an operation against the thread-local dispatcher and register their task waker.

When the executor becomes idle, it submits pending ring entries and polls the ring descriptor through the same reactor wait that handles fd readiness and timer deadlines. The ring descriptor readiness is only a notification that completion queue entries may be harvested. The completed operation still flows through the dispatcher, which matches operation ids, returns owned buffers, and wakes the waiting futures.

The dispatcher tracks:

- queued submissions;
- locally buffered completions;
- registered wakers;

- abandoned operations;
- deferred owned buffers;
- cumulative dispatched, buffered, woken, and discarded operation counters.

5.4 Completion lifecycle

The `io_uring` path is completion-based. A future submits an operation and later receives a completion. That changes the ownership problem. With readiness, the future borrows a stream and retries ordinary system calls. With `io_uring`, the kernel may hold a pointer to an owned buffer until the completion arrives.

The runtime therefore tracks operation state explicitly:

```
future first poll
    allocate operation id
    move owned buffer into dispatcher tracking
    submit operation to ring
    register task waker
    return Pending

completion arrives
    record result and completed buffer
    wake registered task

future next poll
    take completion
    return Ready(result, buffer)
```

Listing 5.2: Simplified completion lifecycle

If the future is dropped before completion, the operation may still exist in the kernel. The dispatcher marks it abandoned and defers buffer cleanup until a completion or safe discard point is observed. This is one of the most important differences between readiness and completion: cancellation is not just removing a waker; it also involves kernel-owned in-flight work.

The dispatcher is intentionally a lifecycle object, not only a syscall wrapper. A thin wrapper around `io_uring_enter` would not be enough for async Rust integration. The runtime needs somewhere to hold buffers, match completions back to futures, wake tasks, and account for futures that vanished before the kernel answered.

5.5 Current limitation

Timers, readiness waits, and `io_uring` completions now share one executor idle wait shape on Linux. This is still a staged integration rather than a production event loop. The runtime does not yet expose a portable completion source, a completion batching policy, or a deeper shutdown drain policy for mixed readiness and completion workloads.

6 Examples

6.1 Learning examples

The `examples/` directory is part of the teaching surface. The examples show the runtime pieces in isolation before combining them:

- basic executor futures and timers;
- readiness-driven TCP helpers;
- sharded executor startup and submission;
- shard-local values;
- CPU placement reporting;
- observability snapshots;
- standard-file and `io_uring` index-building demos.

The examples are deliberately not just smoke tests. Each one is a small experiment with a visible runtime question: what wakes this future, which shard polls it, what state does the snapshot show, or what happens when the work is cancelled? That is why several examples print more internal state than a normal library example would.

6.2 Suggested reading path

The examples are easiest to learn from in layers:

1. Start with `executor_sleep.rs`, `executor_timeout.rs`, and `executor_race.rs`. These show futures, timers, and combinators before sharding enters the picture.
2. Move to `async_readable.rs`, `async_write.rs`, and `async_tcp_pair.rs`. These show readiness and non-blocking file descriptor behavior.
3. Read `async_tcp_server.rs`, `async_tcp_scoped_server.rs`, and `async_tcp_scheduling_groups.rs`. These show handler spawning, scoped shutdown, and scheduling groups.
4. Then read `sharded_executor.rs`, `sharded_submit.rs`, and `sharded_map_reduce.rs`. These show explicit placement of async work on shards.
5. Finally read `shard_local.rs` and `shard_local_worker_observability.rs`. These show local state access and owned runtime snapshots.

Treat examples as executable documentation. Run them, then inspect the modules they use. Most examples intentionally print runtime counters or snapshots so that the mechanism is visible rather than merely implied.

If an example feels a little verbose, that is usually intentional. The output is meant to connect a user-visible event to an internal mechanism. A compact demo that hides the executor state would be less useful for this project.

6.3 Observability examples

`sharded_observability.rs` samples task state while named shard tasks sleep and wake. The interesting columns are:

`status` the coarse task lifecycle state;
`age_ms` how long the task has existed;
`state_ms` how long it has been in the current state;
`polls` how many times the future has been polled;
`wait` the last known wait interest.

`async_tcp_scheduling_groups.rs` shows a different form of observability. It starts two TCP accept loops, parks accepted handlers, takes a snapshot, and prints which scheduling group each live task belongs to. This is a small substitute for a console UI: the snapshot is just owned data, so a tool can render it however it wants.

6.4 Sharded index build

The sharded index example uses two files: a fixed-size record data file and a sorted index of offsets into that data file. Each shard scans one partition, sorts its partition-local index entries, writes a materialized run file, and then participates in merge rounds until one sorted index remains.

The `io_uring` variant performs partition reads, run writes, and merge run reads/writes through the executor-backed Linux completion path. Final verification still uses ordinary file I/O because it checks the externally visible result rather than the runtime path that produced it.

The index examples are intentionally more involved than the small teaching examples. They exercise the Seastar-like direction: split work by shard, keep partition-local state local while sorting, materialize intermediate runs, and merge owned outputs. They are useful for studying where the project is heading, but they are not the first examples to read.

The reason the index build is a good larger example is that it is naturally partitionable. Each shard can sort a portion of the data without sharing a mutable index structure with other shards. The merge phase then makes the coordination cost explicit by exchanging owned run files and summaries rather than borrowing another shard's working set.

6.5 Representative commands

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test
cargo doc --no-deps
```

Listing 6.1: Local validation

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   sharded_index_build_uring -- --records 512 --shards 4 --seed 0xbeef --
   cleanup
```

Listing 6.2: Linux validation through Docker

7 Mechanism Glossary

7.1 Why this chapter exists

The same words appear repeatedly in the runtime: task, shard, waker, readiness, completion, submitter, snapshot, and scheduling group. This chapter is a short field guide for those words. It is not a replacement for the implementation chapters; it is a map to keep beside the code while reading.

7.2 Runtime terms

Shard A single ownership domain. In the standard-library services it is a worker thread with a mailbox. In `ShardedExecutor` it is a thread running one single-threaded executor.

Executor A single-threaded async kernel. It owns tasks, timers, readiness registrations, optional `io_uring` dispatcher state, and scheduler counters.

Task The executor-owned wrapper around a pinned future. A task has an id, optional name, scheduling group id, lifecycle state, poll counters, timestamps, and a waker path back to its executor.

Root future The future passed to `run_until`. The executor drives it while also polling spawned tasks. When the root future completes, `run_until` returns its output.

Spawner A handle that submits new tasks to one executor. Cloned spawners keep the executor accepting work until they are dropped.

Submitter A sharded capability for explicit cross-shard async submission. It owns the ability to place work on shard executors.

Join handle An awaitable handle for a spawned task result. It is also the cancellation handle: aborting it requests that the task be dropped if it has not completed.

Task scope A structured owner for child tasks. It gives a group of children one cooperative stop source and one bounded shutdown path.

7.3 Scheduling terms

Ready queue A queue of tasks that should be polled. In the current executor, each scheduling group owns its own ready queue.

Waker The object a future uses to ask the executor to poll it again. In `sitas`, waking a task requeues it on the same executor.

Poll One call into a future. A future either returns `Ready(output)` or `Pending`. Long polls block the whole executor thread, so cooperative code must yield or await.

Scheduling group An executor-local resource class with a name and relative shares. Tasks in a group are charged for observed poll time.

Virtual runtime A weighted counter used to compare scheduling groups. More shares make the same poll duration charge less virtual runtime.

Ready-poll budget The maximum number of ready tasks polled before the executor checks timers, readiness, and completions again.

7.4 I/O terms

Readiness The operating system reports that a descriptor may be ready for reading or writing. The runtime must still retry the operation and handle partial progress or `WouldBlock`.

Completion The operating system reports that a submitted operation has finished. Linux `io_uring` is completion-based.

Interest A registered wish to be woken when a descriptor becomes readable or writable. Interests are task/waker bookkeeping, not file descriptor ownership.

Dispatcher The executor-local `io_uring` state machine that owns submission tracking, completion buffering, waker registration, and abandoned operation cleanup.

Abandoned operation A completion operation whose future was dropped before the kernel completion was observed. The dispatcher must keep enough state to dispose of buffers safely later.

7.5 Ownership terms

Owned value A value that can safely cross a boundary because no borrowed reference into local runtime or service state escapes with it.

Shard-local value One owned value per shard, accessed only on the owning shard and only through synchronous closures.

Snapshot An owned point-in-time copy of runtime state. A snapshot is not live and does not keep the runtime alive.

Runtime identity A private identity used to reject handles from a different runtime, such as a sharded scheduling group created by another `ShardedExecutor`.

7.6 Mechanism maps

7.6.1 Wake path

```
Future stores Waker
    |
external event or explicit wake
    |
Task waker marks task queued
    |
Scheduler pushes task into its scheduling group queue
    |
```

```
Executor polls ready work later
```

Listing 7.1: Wake path

7.6.2 Timer path

```
sleep(duration)
    |
register deadline and waker
    |
executor waits until earliest deadline or another event
    |
expired timer wakes task
    |
task is requeued and polled
```

Listing 7.2: Timer path

7.6.3 Readiness path

```
read/write future tries non-blocking operation
    |
WouldBlock
    |
register fd interest and task waker
    |
reactor reports readiness
    |
task retries operation
```

Listing 7.3: Readiness path

7.6.4 Completion path

```
io_uring future submits operation with owned buffer
    |
dispatcher tracks operation id, buffer, and waker
    |
kernel completes operation
    |
dispatcher buffers completion or wakes task
    |
future takes result and buffer
```

Listing 7.4: Completion path

7.7 Common confusions

Readiness is not completion Readiness says retrying may work. Completion says submitted work has finished.

Shared-nothing is not synchronization-free The runtime uses locks and atomics internally. The application-state model remains shared-nothing.

A scheduling group is not a thread It is a queue and accounting class inside one executor. On the sharded runtime, a sharded group is one matching local group per shard.

A snapshot is not a subscription It is an owned copy. To observe progress over time, take repeated snapshots and compare them.

CPU placement is not load balancing Placement asks the operating system where shard threads should run. It does not redistribute work between shards.

8 Guided Walkthroughs

8.1 Purpose

This chapter gives concrete source-reading paths. Each walkthrough starts from a visible API or example and follows the mechanism down through the runtime. The goal is to make the code base approachable without flattening the design into prose only.

> **How to use these walkthroughs.** Keep the source tree open while reading. The paths are intentionally explicit. Read the public function first, then follow the listed modules in order.

8.2 Walkthrough: spawning one task

Start with the smallest async unit: a task spawned onto one executor.

```
examples/executor_sleep.rs
src/executor.rs
src/executor/spawner.rs
src/executor/task.rs
src/executor/task_state.rs
src/executor/scheduler.rs
src/executor/task_set.rs
```

Listing 8.1: Task spawn reading path

The public surface is `executor_and_spawner`. It returns one executor and one spawner. The spawner creates tasks; the executor polls them.

The useful questions to ask while reading are:

1. Where is the future boxed and pinned?
2. Where does the task get its id and optional name?
3. What prevents repeated wakes from putting the same task into the ready queue many times?
4. What state is recorded before and after each poll?
5. What happens if the executor is dropped before the task completes?

The mechanism is deliberately local. A task spawned by a `Spawner` is owned by that spawner's executor. Its waker requeues it there. Nothing in this path migrates the task to another thread.

8.3 Walkthrough: sleeping and timers

The timer path starts with examples such as `executor_sleep.rs` and `executor_timeout.rs`. Then read:

```
src/executor/future.rs
src/executor/timer.rs
src/executor/scheduler.rs
src/executor/driver.rs
```

Listing 8.2: Timer reading path

`sleep` creates a future that registers a deadline. When polled before the deadline, the future stores the task waker and returns `Poll::Pending`. The scheduler keeps timer state separate from the ready queues. On each loop iteration the executor checks expired timers and wakes the registered tasks.

`timeout` is useful for learning because it demonstrates cancellation by drop. If the deadline wins, the inner future is dropped. Any timer or readiness registration owned by that future must clean itself up. This is why the runtime has tests for drop behavior rather than only success paths.

8.4 Walkthrough: readiness-driven TCP

Start from the server helpers rather than the raw reactor:

```
examples/async_tcp_server.rs
examples/async_tcp_scoped_server.rs
src/executor/tcp.rs
src/executor/unix_io.rs
src/executor/io_interest.rs
src/os/reactor.rs
```

Listing 8.3: TCP readiness reading path

The TCP helper shows the application shape: accept a connection, spawn a handler, wait for handlers, and propagate errors. The lower layers show the runtime shape: try a non-blocking operation, register interest on `WouldBlock`, sleep in the OS reactor, and wake the task when readiness is reported.

When reading `src/executor/tcp.rs`, notice that the accept loop and the handler tasks are distinct. Grouped TCP helpers use this distinction: the accept loop stays in the caller's current task, while accepted handler tasks are spawned into an explicit scheduling group.

8.5 Walkthrough: scheduling groups

The fastest way to see scheduling groups is to run:

```
cargo run --example scheduling_group_demo -- 1
cargo run --example async_tcp_scheduling_groups
cargo run --example sharded_scheduling_groups
```

Listing 8.4: Scheduling group examples

Then read:

```
src/executor/scheduling_group.rs
src/executor/spawner.rs
src/executor/task_set.rs
src/executor/snapshot.rs
src/sharded_executor.rs
```

Listing 8.5: Scheduling group reading path

`scheduling_group.rs` defines the public handle and its executor ownership. `spawner.rs` validates that a group belongs to the executor before creating a task in that group. `task_set.rs` owns the queues and weighted virtual runtime accounting. `snapshot.rs` resolves group names into task snapshots for observability.

On the sharded runtime, the interesting extra step is runtime identity. A `ShardedSchedulingGroup` is a bundle of executor-local groups, one per shard. It must not be accepted by a different runtime even if names and ids look similar.

8.6 Walkthrough: cross-shard submission

Start with:

```
examples/sharded_submit.rs
examples/sharded_map_reduce.rs
src/sharded_executor.rs
src/sharded_executor/join.rs
src/executor/spawner.rs
```

Listing 8.6: Cross-shard reading path

A cross-shard submit is not a function call into borrowed remote state. It is owned work sent to another executor shard. The submitted future is polled on the target shard. Its owned output completes a join handle. The original task wakes and resumes on its original shard.

While reading the submitter APIs, look for the `Send + 'static` boundaries. They are not incidental. They mark where work can cross threads.

8.7 Walkthrough: shard-local state

Read `examples/shard_local.rs` first, then:

```
src/shard_local.rs
src/sharded_executor.rs
src/runtime.rs
```

Listing 8.7: Shard-local reading path

`ShardLocal<T>` is the clearest example of the shared-nothing idea in the async runtime. Each shard owns one `T`. Access is routed to the owning shard. The closure receives `&mut T` synchronously and returns an owned result.

The important rule is negative: a reference to `T` must not become part of a future that later awaits. The API design enforces this by giving local access through synchronous closures rather than async callbacks.

8.8 Walkthrough: `io_uring` lifecycle

This path is Linux-specific. When available, start with:

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example os_uring
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   os_uring_abandon
```

Listing 8.8: io_uring examples

Then read:

```
src/os/uring.rs
src/executor/uring.rs
src/executor/driver.rs
src/executor/tests.rs
```

Listing 8.9: io_uring reading path

The key distinction is ownership. A readiness future borrows a descriptor and retries ordinary system calls. An `io_uring` future submits owned buffers to the kernel and must keep them alive until completion or safe abandonment cleanup.

`os_uring_abandon.rs` is especially useful because success paths can hide lifetime complexity. Abandonment forces the dispatcher to keep enough state to clean up operations whose futures no longer exist.

8.9 Walkthrough: reading observability output

Run:

```
cargo run --example sharded_observability
cargo run --example shard_local_worker_observability
```

Listing 8.10: Observability examples

The columns are a compact runtime story:

`ready` how many tasks are queued to be polled;
`tasks` how many tasks are still unfinished;
`timers` how many timer registrations exist;
`polls` how many task polls the executor has performed;
`done` how many tasks completed or were cancelled;
`age_ms` how long the task has existed;

`state_ms` how long it has been in the current state;
`wait` the last known wait interest.

The output is not tracing. There is no event stream. It is repeated sampling of owned snapshots. This is less powerful than a full console, but it is simple, portable, and keeps runtime internals inspectable.

8.10 Debugging checklist

When something behaves unexpectedly, ask these questions in order:

1. Which shard owns the state being mutated?
2. Which executor polls the future?
3. Is the task queued, polling, waiting, completed, or cancelled?
4. If waiting, is it waiting for a timer, fd readiness, completion, or an unknown waker?
5. Did a handle from another executor or runtime cross a boundary?
6. If using `io_uring`, who owns the buffer until completion?
7. If shutdown hangs, which spawner, submitter, join handle, or scoped child is still alive?

These questions match the runtime's own invariants. They are usually more productive than starting with performance guesses.

9 Hands-on Labs

9.1 Purpose

This chapter turns the manual into a workbook. Each lab has a command to run, something specific to observe, and a small modification to try. The goal is not to benchmark the runtime. The goal is to connect visible behavior to the mechanisms described earlier.

The labs are organized around "why" questions rather than feature coverage. Why does a timer need a waker? Why does readiness require retrying the system call? Why does cross-shard submission return an owned result? Why does `io_uring` need abandonment tracking? Running the examples first makes those design choices less abstract.

> **Lab style.** Run one lab at a time. After each command, inspect the example source and the runtime module it exercises. The examples are intentionally small enough that changing them should be part of learning them.

9.2 Lab 1: Timers and cooperative polling

Run:

```
cargo run --example executor_sleep
cargo run --example executor_timeout
cargo run --example executor_race
```

Listing 9.1: Timer examples

Observe that the executor can drive futures that suspend and resume without using a third-party runtime. Then read `src/executor/future.rs` and `src/executor/timer.rs`.

Try this:

1. Change the sleep duration in `executor_sleep.rs`.
2. Add a second spawned task that sleeps for a different duration.
3. Print which task completes first.

The lesson is that timers do not run in their own thread. The executor records deadlines, waits until the next interesting event, and wakes tasks whose deadlines expire.

9.3 Lab 2: Wake coalescing

Run the unit tests that mention repeated wakes:

```
cargo test repeated_wakes_share_one_ready_queue_entry -- --nocapture
```

Listing 9.2: Wake tests

Then read the task queueing code in `src/executor/task.rs` and `src/executor/task_set.rs`.

Try this:

1. Add a tiny future that calls its waker multiple times before returning `Pending`.
2. Spawn it and inspect the executor snapshot's ready queue length.
3. Confirm that repeated wake calls do not create repeated ready queue entries for the same task.

The lesson is that a waker requests another poll; it is not permission to flood the scheduler with duplicate queue entries.

9.4 Lab 3: Readiness-driven TCP

Run:

```
cargo run --example async_tcp_pair
cargo run --example async_tcp_server
cargo run --example async_tcp_scoped_server
```

Listing 9.3: TCP readiness examples

Observe the server shape: an accept loop receives a stream, a handler future owns that stream, and shutdown waits for handlers.

Try this:

1. Change a handler to write two chunks instead of one.
2. Add a short sleep between chunks.
3. Watch how the client still observes ordered bytes even though the server task suspends between writes.

The lesson is that readiness is retry-oriented. A writable notification does not mean every byte has been written; the helper must keep retrying until the operation is complete.

9.5 Lab 4: Scheduling groups

Run:

```
cargo run --example scheduling_group_demo -- 1
cargo run --example async_tcp_scheduling_groups
```

```
cargo run --example sharded_scheduling_groups
```

Listing 9.4: Scheduling group demos

Observe the difference between task placement and scheduling accounting. The TCP scheduling example shows which group live handler tasks belong to. The CPU demo shows virtual runtime and charged poll time changing over time.

Try this:

1. Change the shares in `scheduling_group_demo.rs`.
2. Increase the runtime from one second to three seconds.
3. Compare the `executed`, `charged`, and `vruntime` columns.

The lesson is that scheduling groups are cooperative accounting classes, not preemptive priorities. Long-running futures still need to yield.

9.6 Lab 5: Observability snapshots

Run:

```
cargo run --example sharded_observability
cargo run --example shard_local_worker_observability
```

Listing 9.5: Observability examples

Observe the task rows. Focus on `status`, `age_ms`, `state_ms`, `polls`, and `wait`.

Try this:

1. Increase the sleep duration inside the worker tasks.
2. Add one more worker per shard.
3. Observe how task count, timer count, and state duration change.

The lesson is that snapshots are owned samples. They are not tracing events. To build a console-like view, sample repeatedly and compute differences.

9.7 Lab 6: Cross-shard submission

Run:

```
cargo run --example sharded_submit
cargo run --example sharded_map_reduce
cargo run --example sharded_broadcast
```

Listing 9.6: Cross-shard examples

Observe that each example explicitly names where work should run. Nothing migrates implicitly.

Try this:

1. Submit work from shard 0 to shard 1.
2. Have the remote future return `current_executor_shard()`.
3. Assert that the returned shard id is the target shard, not the caller's shard.

The lesson is that the remote future is polled remotely, and the result crosses back as an owned value.

9.8 Lab 7: Shard-local state

Run:

```
cargo run --example shard_local
cargo run --example shard_local_current
cargo run --example shard_local_stoppable_workers
```

Listing 9.7: Shard-local examples

Observe that local values are accessed on owning shards and that workers can be stopped cooperatively.

Try this:

1. Add a second field to the shard-local value.
2. Update it inside `with_current`.
3. Return an owned summary instead of a reference.

The lesson is that shard-local access is synchronous. If you need to await, first produce an owned value and await outside the local-state closure.

9.9 Lab 8: CPU placement

Run:

```
cargo run --example sharded_cpu_placement
```

Listing 9.8: CPU placement example

On Linux, also try the Docker helper:

```
tools/linux-docker.sh cargo run --example sharded_cpu_placement
```

Listing 9.9: Linux CPU placement through Docker

Observe whether placement is applied, unsupported, or rejected. The exact result depends on platform and container CPU permissions.

Try this:

1. Request more shards than the container allows CPUs.
2. Compare best-effort placement with required placement.
3. Inspect the shard snapshots printed by the example.

The lesson is that CPU placement is an explicit request to the operating system. It is not a load balancer.

9.10 Lab 9: Linux `io_uring`

This lab requires a Linux host or container where `io_uring` setup is allowed.

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example os_uring
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   os_uring_batch
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   os_uring_abandon
```

Listing 9.10: io_uring labs

Observe the dispatcher snapshots. Focus on pending submissions, tracked operations, buffered completions, registered wakers, and abandoned operations.

Try this:

1. Increase the number of submitted operations in `os_uring_batch.rs`.
2. Drop a future before completion in a small variant of the abandon example.
3. Watch how abandonment and deferred buffer counters change.

The lesson is that completion-based I/O has a different cancellation problem than readiness-based I/O. The kernel may still own an operation after the future is gone.

9.11 Lab 10: Sharded index build

Run:

```
cargo run --example sharded_index_build -- --records 512 --shards 4 --seed 0
   xbeef --cleanup
```

Listing 9.11: Sharded index build

On Linux with `io_uring`:

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
    sharded_index_build_uring -- --records 512 --shards 4 --seed 0xbeef --
    cleanup
```

Listing 9.12: io_uring index build

Observe how partition work is spread across shards, then merged into one final sorted index.

Try this:

1. Change the record count.
2. Change the shard count.
3. Compare per-shard partition summaries and final verification output.

The lesson is that the runtime's value appears when a problem can be split into owned shard-local work, then combined through explicit owned results.

9.12 After the labs

After running these labs, return to the walkthrough chapter and read the modules again. The source is easier to understand after seeing the runtime's observable behavior. If a lab result is surprising, use the debugging checklist from the walkthrough chapter before changing code.

10 Operations

10.1 Building this manual

The manual lives in `docs/system`. Build it with:

```
cd docs/system
make
```

The output is `docs/system/sysdoc.pdf`. The build uses `pdflatex` twice so the table of contents and references settle.

If the build fails, first check that the LaTeX distribution has the standard packages used by `sitasdoc.cls`: `geometry`, `fancyhdr`, `titlesec`, `titletoc`, `listings`, `xcolor`, and `hyperref`. The manual intentionally avoids project-specific generated assets so it can be rebuilt from text alone.

Keeping the manual build simple is intentional. The documentation should not depend on the runtime being runnable, Docker being available, or generated diagrams being checked out from another tool. A plain text-to-PDF path makes it cheap to update design explanations in the same commits as code changes.

10.2 Cleaning generated files

```
cd docs/system
make clean
make distclean
```

`make clean` removes LaTeX auxiliary files. `make distclean` also removes the generated PDF.

10.3 Linux validation

Some runtime behavior is Linux-specific. The Docker helper mounts the project directory into a Linux container and can request an `io_uring`-capable configuration:

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test os::uring::tests --
    --nocapture
```

When the host or container configuration cannot provide `io_uring`, the Linux-only examples report that the backend is unavailable rather than faking a successful run.

The Docker path exists because macOS is a good development environment for most of the code, while Linux is the only honest place to test Linux primitives such as hard CPU affinity and `io_uring`. The helper keeps that platform check repeatable without pretending the project is platform-neutral in areas where it is not.

10.4 Validation strategy

Use narrow tests first, then broaden. This keeps iteration fast while still protecting the runtime invariants.

```
cargo test task_snapshot -- --nocapture
cargo test scheduling_group -- --nocapture
cargo test serve_tcp -- --nocapture
cargo fmt --check
cargo test
cargo clippy --all-targets -- -D warnings
cargo doc --no-deps
```

Listing 10.1: Typical focused-to-broad workflow

For changes touching Linux-only code, use the Docker helper after local macOS validation:

```
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test
SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo clippy --all-targets -- -D
    warnings
```

Listing 10.2: Linux platform pass

The order matters. Focused tests make it easier to understand the mechanism that failed. Broad tests then protect against regressions in adjacent runtime paths. This is especially important in executor code, where a small lifecycle change can affect timers, readiness, joins, and shutdown at the same time.

10.5 Interpreting unsupported features

Unsupported is a valid result when the platform cannot provide a feature. CPU placement is unsupported on non-Linux platforms. `io_uring` may be unavailable if the host kernel, container runtime, or security profile blocks ring setup. The project should report those cases honestly instead of hiding them behind a fake success path.

This matters for learning. A runtime experiment should make platform boundaries visible: readiness works on several Unix backends, hard CPU affinity is Linux-specific, and `io_uring` is both Linux-specific and dependent on host/container permissions.